



SCHOOL OF COMPUTER SCIENCE

ASyMphony: Making Assembly Engaging with an Integrated Live Coding Platform

Finn Cooper

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree
of Master of Engineering in the Faculty of Engineering **worth 40CP**.

Tuesday 5th May, 2026

Abstract

Assembly programming and computer architecture are often difficult topics to teach because much of the behaviour that learners need to understand is hidden. This is reflected in the long standing use of simplified simulations in architecture education, such as the Little Man Computer (LMC) paradigm [36]. However, these tools remain primarily observational. In contrast, Playsemble, a recent assembly programming tool, makes the consequences of low-level code more apparent by relating them to actions in a game world [35]. Musical live coding offers a related opportunity by making program behaviour audible and immediately exposing the effects of code to programmers.

This dissertation proposes that integration with a musical live coding system can make assembly education more usable and engaging. It documents the design, implementation and evaluation of ASyMphony, an assembly-like live coding language and integrated browser environment. Programs run on a virtual machine with multiple execution cores, registers, and shared memory that are explicitly exposed in the interface. Source code highlights indicate which lines are executing. Register and memory access feedback shows which machine components are being modified.

The project was developed in two iterations. First, a minimum viable product (MVP) tested whether a small assembly-like language could control musical output while exposing registers and memory. The final product addressed the MVP's accessibility and usability issues by implementing a four-core machine model, clearer timing semantics, compiler typing and diagnostics, and a guided tutorial.

Evaluation used heuristic analysis, think-aloud feedback, the System Usability Scale (SUS), and a final Likert-scale questionnaire. The average SUS score improved from 63.54 for the MVP to 81.04 for the final product across twelve responses. Final questionnaire results were most positive for user engagement, musical relevance, and perceived usefulness as an introduction to assembly concepts. ASyMphony's usability and engagement are therefore demonstrated well, but evidence is weaker for objective learning gains and musical expressiveness.

Dedication and Acknowledgements

I would like to thank my supervisor Pete Bennet for his support and enthusiasm towards the project, everyone who tested ASyMphony in both its MVP and final form for providing valuable feedback, and my friends and family for their never-ending encouragement throughout my degree.

Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Finn Cooper, Tuesday 5th May, 2026

AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.

Finn Cooper, Tuesday 5th May, 2026

Contents

1	Introduction	1
1.1	Context	1
1.2	Motivation	1
1.3	Aims and Objectives	2
2	Contextual Background	3
2.1	Musical Live Coding	3
2.1.1	Relevant Live Coding Systems	3
2.2	Music Programming in Education	5
2.3	Computer Architecture and Assembly Education	5
2.3.1	Gamified Low-Level Learning	6
2.4	Summary	6
3	Technical Background	7
3.1	Computer Architecture	7
3.2	Virtual Machines	7
3.3	Instruction Set Architectures	8
3.4	Compilation and Assembly	8
3.5	Browser Technologies	9
3.5.1	CodeMirror	9
3.5.2	Lezer	9
3.5.3	Browser Audio	9
4	Methodology and Requirements	11
4.1	Methodology	11
4.2	Design Requirements	11
4.2.1	Accessibility	11
4.2.2	Language	11
4.2.3	Musicality	12
4.2.4	Transparency	12
4.3	Framework Decisions	12
4.3.1	Execution Model	12
4.3.2	Browser Application Stack	12
4.3.3	Deployment	12
4.3.4	Editor and Parser Tooling	13
4.3.5	Audio Runtime	13
5	Minimum Viable Product	14
5.1	Design	14
5.1.1	Execution Model	14
5.1.2	Instruction Set	14
5.1.3	User Interface	15
5.2	Implementation	16
5.2.1	Audio Engine	16
5.2.2	Grammar	16
5.2.3	Compiler	16
5.2.4	Runtime	17

5.3	Reflection	18
5.4	Evaluation	18
5.4.1	Usability Heuristic Analysis	18
5.4.2	Think-Aloud Sessions	19
5.4.3	Exit Survey	20
5.5	Summary	20
6	Final Product	22
6.1	Design	22
6.1.1	Multi-Core Separation	22
6.1.2	Timing and Transport	22
6.1.3	Registers and Memory, and Instruction Set	22
6.1.4	Error Feedback	23
6.1.5	Tutorial System and Sample Map	23
6.2	Implementation	25
6.2.1	Runtime Architecture	25
6.2.2	Compiler and Diagnostics	25
6.2.3	CPU, Cores, and Memory	25
6.2.4	Highlights	25
6.2.5	Scheduling	25
6.2.6	Tutorial	25
6.3	Reflection	26
6.4	Evaluation	26
6.4.1	Usability Heuristic Analysis	26
6.4.2	System Usability Scale	28
6.4.3	Questionnaire	28
6.4.4	Summary	29
7	Critical Evaluation	30
7.1	Overview	30
7.2	Limitations	30
7.2.1	Heuristic Analysis	30
7.2.2	Questionnaire	30
7.2.3	Musical Expression	31
7.2.4	Design Trade-Offs	31
7.3	Evaluation Against Objectives	31
7.3.1	O1: Accessibility	31
7.3.2	O2: Conceptual Transparency	31
7.3.3	O3: Musical Engagement	32
7.3.4	O4: Musical Expressiveness	32
7.3.5	O5: Educational Viability	32
8	Conclusion	33
8.1	Summary	33
8.2	Contributions	33
8.3	Future Works	33
A	AI Prompts/Tools	37
B	Auxiliary Material	38

List of Figures

2.1	An example Sonic Pi <code>live_loop</code> from [27].	3
2.2	Example strudel code from [28].	4
2.3	Example gibber code from [9].	4
2.4	Example chuck code from [7].	5
2.5	Example ORCA code from [10].	5
5.1	Overview of the MVP interface showing code editor with syntax highlighting and line highlights on the left, and controls, registers, and memory on the right.	16
6.1	View of one core showing accurate syntax error diagnostics.	23
6.2	View of one core showing the result of a runtime fault.	24
6.3	Screenshot of the tutorial interface showing the ‘task’ step of the first lesson.	24
6.4	Final interface in its running state, showing four cores, per-core registers, global controls, shared memory, the drum note map, register and memory highlighting and source code highlights.	26
B.1	Results from engagement questions in the final questionnaire.	39
B.2	Results from perceived learning questions in the final questionnaire.	40

List of Tables

5.1	The opcodes, operands, and functions of instructions present in the MVP.	15
5.2	The syntax and function of additional language constructs.	15
5.3	ASyMphony's MVP grammar in Extended Backus-Naur form, where 'letter' and 'digit' are trivial.	17
5.4	MVP evaluation against the implementation requirements.	21
6.1	Comparison of issue severity for each usability heuristic between the MVP and final product.	27
6.2	Final questionnaire results, scored from 1 (strongly disagree) to 5 (strongly agree). The audio-distraction item is negatively framed.	28
6.3	Final product evaluation against the implementation requirements.	29

Ethics Statement

This project fits within the scope of ethics application 6683, as reviewed by my supervisor, Pete Bennett.

Supporting Technologies

- **React and Next.js:** used to implement the browser-based user interface, page structure, and static deployment artefact.
- **Tailwind CSS:** used to enforce consistent visual styling throughout the interface.
- **CodeMirror:** used as the integrated code editor, including keybind handling, syntax highlighting, and diagnostics.
- **Lezer:** used to define ASyMphony’s grammar and generate a parser for compiler and editor support.
- **Git and GitHub:** used for version control, providing public source code, and deploying the application.

Notation and Acronyms

API	:	Application Programming Interface
BPM	:	Beats Per Minute
CPU	:	Central Processing Unit
ISA	:	Instruction Set Architecture
LMC	:	Little Man Computer
MVP	:	Minimum Viable Product
MIDI	:	Musical Instrument Digital Interface
OSC	:	Open Sound Control
UI	:	User Interface
VM	:	Virtual Machine

Chapter 1

Introduction

1.1 Context

Programming and music are closely related in that they both concern ordered structures that evolve over time. Programs specify state changes and processor actions, while music arranges sound, silence, repetition, variation. Musical live coding makes this relationship explicit by allowing performers to write and change music-generating code while it is executing. In live coding, code is part of the performance – the audience can see the program evolve and hear its effects in real time.

This relationship has already been utilised successfully in educational programming environments. Sonic Pi and EarSketch use music to motivate introductory programming, giving learners immediate audible results from short programs [1, 13]. Scratch demonstrates the wider educational value of creative programming environments by lowering syntactic hurdles and making program structure visible [22]. At the other end of the live coding spectrum, systems such as TidalCycles, Strudel, and Gibber support expressive real-time composition, but are primarily concerned with patterns, synthesis, and musical performance [14, 28, 24]. These examples show a common accessibility strategy. Beginner-facing environments usually raise the level of abstraction so that learners can get stuck in before having to think about low-level machine state.

Assembly programming sits in a very different educational space. It exposes the programmer to registers, memory, instruction execution, addressing, and control flow. These are important concepts because they underpin how software ultimately runs, but are often alien to those who are beginning to program with high-level languages.

This dissertation addresses the gap between these two traditions through the development of ASyMphony, a browser-based live coding environment in which users write a deliberately small assembly-like language to produce music. Instead of hiding the underlying execution mechanism, ASyMphony makes it visible through exposed cores, registers, shared memory, source-code highlights, and runtime feedback. The project therefore treats musical sequencing not only as a creative task, but also as a education tool for introducing assembly-level ideas. ASyMphony capitalises on the similarities between music composition and programming to make low-level execution engaging and approachable by allowing users to hear the audible consequences of their programs in a creative way.

1.2 Motivation

Low-level computing is often difficult to teach because its ideas involve dynamic processes that are hard to visualise from lecture- and textbook-based presentation alone. Registers change rapidly, memory is abstract, and control flow is not directly observable. Computer architecture education has long used simplified machines and simulators to make these ideas easier to inspect. Tools that follow the Little Man Computer paradigm use real-world analogies alongside visible memory, an accumulator, and a program counter to make stored-program machines understandable [36]. Playsemble, a recent educational tool, connects assembly programming to a game world, allowing register and memory changes to affect a visible interactive outcome [35]. Its deployment in an undergraduate computer architecture course with 107 students is particularly relevant: the authors report that game-based assembly tasks promoted active experimentation, sustained engagement, and deeper conceptual understanding. This suggests that low-level programming becomes approachable when abstract machine operations are attached to consequences that users care about.

ASyMphony explores whether musical feedback can be a driver of engagement and approachability. In a live-coding setting, program behaviour is not merely inspected after the fact; it takes place in real time and can be heard immediately. Repetition becomes looping, arithmetic becomes transformation, and control flow becomes musical structure. This hands-on interaction with immediate feedback can reduce the distance between cause and effect that may be present for beginners and therefore widen participation.

The audience for this project is therefore not only experienced assembly programmers. It is most useful for learners who are early in computer architecture or low-level programming and need a more approachable first encounter with registers, memory, program counters, and jumps. It may also benefit creative programmers, musicians, and live coders who are comfortable experimenting with code but have little reason to engage with assembly concepts in their normal tools. For educators, the project offers a usable classroom tool – students can be asked to predict state changes, modify loops, and hear the effect of different instructions.

Accessibility is also important. Some established live coding systems are powerful but have non-trivial setup requirements. TidalCycles depends on several external software components, including the Haskell language, SuperCollider, SuperDirt, and text editor integration [30]. Strudel shows an alternative direction by bringing the established pattern-based ideas of TidalCycles, along with comprehensive documentation and an interactive tutorial, to the browser [26, 28]. ASyMphony follows this browser-first approach so that the programming environment, execution visualisation, tutorial material, and audio output can live in one place without specialist setup.

Designing such a system introduces several challenges. The language must remain small enough for novices while still allowing for ample musical expression. The execution model must expose registers, memory, and control flow clearly without overwhelming users. Musical timing must be handled carefully so that non-musical instructions do not distort the produced rhythm. Finally, the interface must make machine state understandable at a glance if the system is to function as both a creative tool and a learning environment. These challenges are not independent: decisions about language design, timing, and interface visibility all influence whether the environment feels understandable or bewildering.

1.3 Aims and Objectives

The central aim of this project is to investigate whether assembly-level programming can be made more approachable and engaging by packaging it in a musical live coding environment. This aim is divided into five broad objectives, which are later evaluated against in Chapter 7:

1. **O1: Accessibility:** reduce setup, syntax, and interface barriers so that users can begin experimenting without installing specialised software.
2. **O2: Conceptual Transparency:** make system execution easier to understand through inspectable registers, memory, control flow, timing, and feedback.
3. **O3: Musical Engagement:** use live musical output to make assembly-style programming feel motivating, responsive, and worth exploring.
4. **O4: Musical Expressiveness:** expose enough musical control to support the creation of complex musical pieces.
5. **O5: Educational Viability:** produce a tool that is suitable for the introductory teaching of assembly concepts.

Chapter 2

Contextual Background

This chapter establishes the existing landscape of live coding and low-level computing education. It begins with musical live coding and several systems that make code audible and visible during performance, then looks at educational music programming environments that use sound to support learning. Finally, it examines assembly and computer architecture education, where simplified simulators, visible components, and interactive feedback are used to make low-level execution understandable.

2.1 Musical Live Coding

Live coding is the creation or transformation of a running system by editing its code in real time. In musical live coding, the running system produces sound. A programmer writes code, evaluates it, hears the result, and modifies the program while the music continues. This turns programming into a feedback loop, where changes can be tested immediately.

Visibility is central to live coding culture. During performances, code is often shown to an audience so that the musical process is exposed rather than hidden. The TOPLAP manifesto captures this expectation through its demand that code should be “seen as well as heard” [32]. McLean similarly frames live coding as a performance practice concerned with liveness, temporality, and visible program structure [14]. This code visibility matters because it lets audiences and performers relate changes in the written program to changes in sound.

2.1.1 Relevant Live Coding Systems

Sonic Pi

Sonic Pi was developed by Sam Aaron and collaborators as a tool for computing and music education [1]. It extends the Ruby programming language with keywords like `play`, `sleep`, `sample`, and `live_loop` that allow for the looped playback of both synthesised and sampled sounds [27]. Its educational design is important. The system was developed through educational partnerships, rather than only as a performance tool. This makes it a strong example of music being used as a route into programming. It also shows the value of language design that uses familiar musical terms while still allowing users to learn fundamental programming structures like loops, functions, and state manipulation.

```
live_loop :flibble do  
  
  sample :bd_haus, rate: 1  
  
  sleep 0.5  
  
end
```

Figure 2.1: An example Sonic Pi `live_loop` from [27].

TidalCycles and Strudel

TidalCycles is a live coding language written in Haskell [14]. Its central idea is that musical patterns can be represented as functions over time. Instead of placing notes on a timeline directly, the performer manipulates patterns which are queried over time to produce musical events. This makes Tidal powerful for representing repetition, variation, and polyrhythm. However, this strength makes it less approachable for beginners. A Tidal setup requires the installation of Haskell, SuperCollider, SuperDirt, and a text editor with a compatible Tidal plugin [30]. This might be reasonable for experienced performers, but it creates a significant entry cost for someone who is only beginning to explore live coding.

In contrast, Strudel is a web-based version of the TidalCycles pattern language, designed to be accessible to anyone with a web browser [26, 28]. It outputs audio directly in the browser by default using the Web Audio API, but can also output MIDI or OSC commands to drive connected virtual or physical instruments [26]. This design removes much of the installation friction, and demonstrates that a browser can combine code editing, examples, documentation, and audio output in one interface.

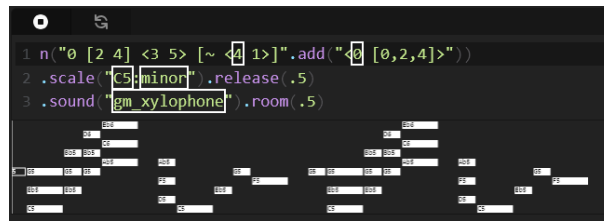


Figure 2.2: Example strudel code from [28].

Gibber

Gibber is another web-based live coding system [24, 25]. It is an audio-visual programming environment written in JavaScript, and again demonstrates that the browser can be used as a capable live coding medium. Its interface combines source code with highlights and visualisations that show what the program is doing over time. This reinforces the idea that system transparency is important in the live coding user experience.

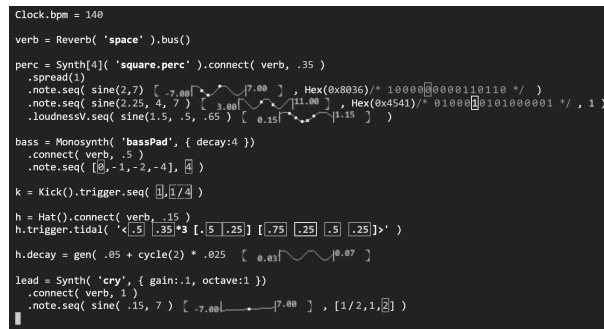


Figure 2.3: Example gibber code from [9].

ChuckK

ChuckK is a strongly timed live programming language that is built around a virtual-machine model and precise control of musical time [34]. Time and duration are native types in ChuckK's language, and musical time must be explicitly advanced by manipulating the `now` keyword [6]. This directly exposes the relationship between computational and musical time to the programmer, allowing for fine-grained control over the resulting musical structure.

ORCA

ORCA is an unconventional live coding language in that it uses a two-dimensional grid of characters as the input and execution field [10]. Characters act as operators that read from neighbouring cells and


```
1 /* Play a sine wave at 440Hz for 1 second */
2 SinOsc osc => dac; // create an oscillator
3 440 => osc.freq;
4 0.2 => osc.gain;
5 now + 1::second => time later; // define a time in the future
6 later => now; // advance current time
```

Figure 2.4: Example chuck code from [7].

produce events over time. It does not produce its own sound, but instead generates MIDI, OSC, or UDP messages that control external physical or virtual devices. ORCA is useful as an example of a language where spatial structure and visible execution are part of the programming model.

[illegible]

Figure 2.5: Example ORCA code from [10].

2.2 Music Programming in Education

Music programming is educationally attractive because it gives immediate feedback and creates meaningful output. A learner can hear that a program has changed even before they can fully explain the underlying code. This makes music a useful context for exploratory learning: changing a number, command, or loop can produce an audible difference. Sonic Pi and EarSketch show how music can motivate introductory programming by letting users write code that produces tangible output [1, 13]. Scratch demonstrates the wider educational value of creative programming environments by reducing syntax barriers and making program structure visible [22].

Music gives learners a reason to experiment with repetition, variation, timing, parameter changes, and structure. These are computational as well as musical ideas. It is therefore possible for sound to act as immediate feedback for program behaviour, while the creative task of music-making helps keep learners engaged.

2.3 Computer Architecture and Assembly Education

Due to the complex nature of real processors, computer architecture education often relies on simplified machines and visual simulators. These models can expose memory, registers, a program counter, and an instruction set without requiring students to understand the full details of hardware implementation. This makes otherwise abstract behaviour directly inspectable.

The Little Man Computer (LMC) paradigm is a classic example of a simplified educational machine model [36]. It represents a computer as a post room with numbered storage locations, an accumulator, an inbox and outbox, and a program counter. Programs and data are stored in the numbered locations, and learners can trace execution by observing how the accumulator, program counter, and memory change. The strength of LMC is that it uses a real-world analogy and explicitly visible state to make the behaviour of a stored-program computer tangible while still preserving important architectural concepts.

2.3.1 Gamified Low-Level Learning

Playsemble is a recent browser-based tool that teaches low-level programming by asking students to write assembly code that controls a Pac-Man-like character inside a video game environment [35]. The interface combines an editor, game world, visible registers and memory, integrated debugging, and structured tasks. These tasks introduce progressively more complex low-level concepts through an interactive goal; register and memory changes matter because they affect character movement and game behaviour. The important lesson from Playsemble is that low-level execution becomes easier to reason about when visible machine state is connected to meaningful interactive consequences. This is a useful design principle for low-level learning tools generally, regardless of the feedback medium.

2.4 Summary

Several themes emerge from the contextual background. Live coding systems show the value of immediate feedback, code visibility, and runtime modification. Music programming environments show that immediate audio feedback can make programming more engaging and exploratory. Architecture education tools that show registers, memory, program counters, and instruction execution become easier to understand when exposed visually. The success that Playsemble finds with the gamification of low-level learning shows that visible state is more powerful when connected to meaningful outcomes.

Chapter 3

Technical Background

This chapter introduces the technical concepts needed to understand the later design and implementation chapters. It begins with the ideas behind low-level programming, with computer architecture, instruction sets, virtual machines, and compilation.

3.1 Computer Architecture

Computer architecture describes the abstract machine model on which software executes. Although the physical design of modern central processing units (CPUs) is highly complex, a small number of ideas remain central at an abstract level: memory stores data over time, registers hold values needed immediately by the processor, and execution consists of operations that transform the state of the machine [19]. These concepts are widely used not only in hardware design, but also in programming language implementation, virtual machines, and educational models of computation.

Registers

Registers are small amounts of storage located directly within the CPU. Due to their close proximity to physical execution units, registers can be accessed extremely quickly and typically hold temporary values, counters, addresses, and other data needed immediately during program execution.

Main Memory

Main memory is the primary location where data is stored during program execution. It is much larger in capacity than registers and is used for values that must persist beyond a single operation.

In simplified architectural models, memory is often represented as an indexed array of locations, each of which can be read from or written to. This abstraction is particularly useful in computing education because it makes the state of the machine straightforward and allows the effects of computation to be easily traced over time.

Multi-Core Systems

Modern CPUs almost always contain multiple cores, each capable of executing a stream of instructions in parallel. Multi-core systems therefore form an important part of the modern computing landscape, and they have influenced the design of programming models, operating systems, and software tools that must coordinate concurrent processes.

3.2 Virtual Machines

Virtual machines (VMs) provide an abstract execution environment implemented in software. A VM can define its own registers, memory, instruction set, and input/output behaviour. This allows a machine model to be designed for a specific purpose rather than simply inherited from underlying hardware.

VMs are useful in programming language implementation because they provide a stable target for compilation or interpretation. Instead of translating source coding directly into physical machine code, a compiler can translate it into instructions for an abstract machine. They are also useful in education

because their state is implemented at the software-level. It can be inspected, displayed, and manipulated more easily than the state of a real hardware processor.

3.3 Instruction Set Architectures

An instruction set architecture (ISA) is the abstract interface between software and a processor. It defines the operations a machine can perform, the form in which instructions are represented, and the computational resources those instructions can access such as registers, and memory. The ISA therefore provides a vocabulary for computation: it determines how programs express arithmetic, data movement, and control flow, without requiring direct concern for the physical details of the underlying hardware implementation.

Opcodes and Operands

Instructions in an ISA represent a discrete operation such as loading a value, storing data, performing arithmetic, or changing the flow of execution. In modern ISAs like RISC-V, they consist of an opcode, which identifies the operation to be performed, and one or more operands, which specify the values, registers, or memory locations involved [23].

Opcodes have an expected number and type of operands. For example, a three-operand arithmetic instruction may use one destination operand and two source operands: `add r0, r1, r2` adds the content of `r1` and `r2` and places the result in `r0`. A two-operand form, such as `add r0, r1`, uses the first operand as both a source and a destination. This is less general, but potentially easier to explain because it corresponds to the accumulative idea of adding one value to another in-place.

Addressing and Data Access

ISAs also define how instructions refer to data. Some operands may represent immediate values embedded directly within an instruction, while others may refer to registers or to locations in memory. These are the addressing modes of the ISA.

With direct register or immediate addressing, the operand itself supplies the value. With memory addressing, the operand refers to a location in memory which the value must be read from or written to. Indirect addressing adds another level – a register operand contains the address of the memory location to be accessed. This is commonly used for pointers and iteration through stored data.

Control Flow

Control-flow instructions change which instruction will execute next, disrupting the usual program order. Unconditional jumps set the program counter to a new address directly. Conditional jumps do so only if a condition is met. Even in a small educational ISA, control-flow instructions are important because they introduce the idea that current program state can influence the path of future execution.

3.4 Compilation and Assembly

Programming languages are typically not executed directly in the form written by the programmer. Instead, source code is transformed through a sequence of stages into a representation that can be executed by a targeted machine [2]. Although the exact structure of this process varies between systems, the general idea is that higher-level textual input is analysed, checked, and translated into a lower-level form that is more suitable for execution.

Lexing and Parsing

One of the first stages in language processing is the analysis of the source text itself. This typically begins with *lexing*, in which a stream of characters is grouped into meaningful units such as keywords, identifiers, numbers, and symbols [2]. These units are referred to as tokens. Once tokenised, the input can be *parsed*, meaning that the token sequence is checked against the grammar of the language in order to determine its structure.

Parsing is important because it moves beyond the raw text of a program and identifies how that text is organised. Rather than treating code as a flat sequence of characters, the parser determines which

parts of the input form instructions, expressions, declarations, or other syntactic structures. This makes parsing a central stage in almost any compiler, assembler, or language-oriented tool.

Syntax Trees

The result of parsing is commonly represented as a tree structure. In such a representation, the hierarchical structure of the source code becomes apparent. Larger constructs contain smaller ones, and the relationships between them can be inspected programmatically.

Structured representations are useful because later stages of the pipeline usually operate on them rather than on raw text. Error reporting, syntax highlighting, semantic checking, optimisation, and code generation all depend on some representation of program structure.

Assembly

After the syntactic structure of a program has been identified, the next stage is to translate that structure into a form suitable for execution. In some systems this first produces an intermediate representation, which can then be further optimised before execution. In others, the result may be an assembly-like instruction sequence or a directly executable binary form.

Assembly code is usually associated with low-level symbolic instructions that correspond closely to machine operations. An assembler resolves symbolic names and converts these instructions into a more concrete representation. In virtual-machine based systems, assembly may instead target a virtual instruction set rather than physical hardware.

3.5 Browser Technologies

3.5.1 CodeMirror

CodeMirror is a code editor component for user interfaces that is lightweight but extensible. For developers, it exposes an editor state, a visible editor view, and an extension system through which features such as syntax highlighting, linting, autocompletion, and custom update handling can be added [8]. This makes it a useful foundation for building language-specific editing environments without having to implement a text editor from scratch.

3.5.2 Lezer

Lezer is a parser system for JavaScript-based language tools. It allows developers to define a grammar for a language and generate a parser that can recognise and parse source code written in that language [11]. Given an input program, it can produce a syntax tree describing how the text is organised according to the grammar. The generated syntax tree is well integrated with CodeMirror's language extensions, making it easy to define a custom grammar with Lezer and implement syntax highlighting, autocompletion, and diagnostic feedback in CodeMirror's editor.

3.5.3 Browser Audio

Web-based audio systems can be built at several levels of abstraction. Basic browser audio playback can be achieved through media elements, but more interactive systems require lower-level control over when sounds occur, how they are generated, and how they are processed. For this reason, browser-based music and audio applications are commonly built either directly with the Web Audio API or wrapper libraries like Tone.js that provide more convenient abstractions for sequencing, synthesis, and sample playback [31].

The Web Audio API is the main browser-native system for programmable audio. It is organised around an `AudioContext` containing a graph of connected `AudioNode` objects, where nodes can represent sources, outputs, or intermediate processing stages [33]. This model is important because it allows audio in the browser to be treated as a programmable signal chain rather than as simple media playback, making it suitable for applications that need to control complex audio output.

Scheduling and Musical Timing

A central challenge in browser-based music systems is not only producing sound, but producing it at the correct time. For this reason, many higher-level libraries focus on timing and sequencing abstractions built on top of the native audio engine. Tone.js is a notable example: its documentation presents a transport model for musical timing, with support for synchronising multiple events and passing the precise scheduled time into callbacks [31]. Still, the Web Audio API does expose an accurate audio clock via `currentTime` which allows for precise audio scheduling [33].

Chapter 4

Methodology and Requirements

This chapter turns the background material into an implementation method, a set of requirements, and architectural decisions. importance of requirements.

4.1 Methodology

ASyMphony was developed in two stages: a minimum viable product (MVP) and a final product. This split was chosen to allow user feedback to be collected earlier in the process, which could steer later development towards fixing real user-facing issues and shaping design to align with user needs.

Each iteration was evaluated against the requirements introduced in Section 4.2. The MVP evaluation focused on whether the central concept was clear and usable, while the Final Product evaluation assessed whether the refined system addressed the MVP’s issues and met the project aims.

4.2 Design Requirements

4.2.1 Accessibility

Related work shows that ease of access matters. Strudel’s browser-first design lowers the setup cost of Tidal-style live coding and integrates code editing, audio output, and an interactive tutorial within a single interface. Sonic Pi shows the value of tools that are approachable in educational settings [26, 28, 1].

- R1** The system should be publicly accessible, without requiring installation of specialist music or compiler software.
- R2** The interface should combine code editing, program execution, audio output, and runtime visualisation in a single environment.
- R3** The system should provide sufficient examples, help text, or tutorial material for a new user to begin editing and running programs without external documentation.

4.2.2 Language

Assembly education tools like LMC are effective because they reduce complexity while preserving important low-level ideas. In order for ASyMphony to provide transferable assembly programming skills, its language must remain small enough to learn quickly but concrete enough to support assembly-level reasoning.

- R4** The language should use a compact instruction set with short, memorable opcodes and a consistent operand structure.
- R5** The language should support register and memory modification, loops, and conditional behaviour.
- R6** Each instruction should have a clear computational or musical role.

4.2.3 Musicality

Live coding environments are not only programming tools, but also musical instruments. To succeed as a live coding environment, ASyMphony must allow for non-trivial musical control and immediate live feedback during execution.

R7 The environment should support the output of multiple simultaneous musical parts.

R8 Musical parameters such as volume, panning, duration, and timbre should be controllable using the same model as the computational behaviour of the machine.

R9 The system should allow programs to be edited and reloaded during playback.

4.2.4 Transparency

Educational architecture tools demonstrate the importance of visible state. An educational live coding machine must not only produce audible output; it must show why the output happened.

R10 Register contents, memory cells, program position, playback status, and code errors should be visible.

R11 Parallel execution should be made explicit to maintain the relationship between cause and effect.

R12 Source code, machine state, and audio output should be visibly connected.

4.3 Framework Decisions

4.3.1 Execution Model

The requirements make a direct audio-command interpreter unsuitable. A system that simply converted each line of code into an immediate sound would not provide a coherent place for registers, memory, program counters, faults, or parallel execution. The language therefore needed an explicit target machine whose state could be inspected, scheduled, and visualised.

For this reason, ASyMphony implements a small virtual machine. Source code is compiled into instructions for an abstract machine, and the runtime steps that machine in accordance with the beat. This follows the broader idea behind strongly timed audio systems such as ChucK, where a virtual-machine model coordinates program execution with musical time [34]. In this project, the VM also serves an educational purpose: it makes machine state a direct part of the interface rather than an implementation detail hidden behind audio playback.

4.3.2 Browser Application Stack

Requirements **R1-R3** motivated a browser-based implementation. A desktop application would have allowed deeper control over audio and file-system integration, but it would also have introduced installation friction. A browser application can be opened immediately, shared easily for testing, and used in teaching or demonstration settings without asking participants to install specialist tools.

The implementation used the Next.js framework, which combines TypeScript, React, and Tailwind CSS, along with static browser application generation and clear page routing [15, 20, 29]. TypeScript reduced risk in the development of the execution model compared to JavaScript by making the interaction between compiler and runtime components explicit with its strong typing. React was chosen due to the need for repeated stateful UI components, as outlined by requirements **R2**, **R3**, **R10**, and **R12**. Tailwind CSS made it practical to build a dense and consistent custom interface without maintaining a separate large stylesheet for every possible state of a component.

4.3.3 Deployment

GitHub provided a means of application deployment that made the latest version of ASyMphony accessible to anyone on the web at any time. The application was configured as a static Next.js export and deployed through GitHub Pages, which matched the requirement for a lightweight browser application that could be shared and accessed without installing local tools.

4.3.4 Editor and Parser Tooling

Requirements **R4-R6** and **R10-R12** indicate that plain text input would not be sufficient. The language needed recognisable syntax, source code diagnostics, runtime code highlighting, and agreement between source and compiled code. Those features depend on source code positions and syntactic analysis.

CodeMirror was chosen as the code editor because its modular extension system supports syntax highlighting, linting, autocomplete, and code decorations for custom languages [8]. The main alternative considered was Monaco, Visual Studio Code’s editor; however, a comparison of browser editors presents Monaco as a bulkier option and CodeMirror as easier to customise [21]. Choosing CodeMirror also allowed for integration with Lezer, a parser generator from the same ecosystem. Lezer produces a structured syntax tree from a defined grammar that can be used for both code editor features and compiler traversal [11]. This meant that syntax highlighting, compile-time diagnostics, and runtime source highlights were all tied to the same authoritative tree structure.

4.3.5 Audio Runtime

Requirements **R7-R9** and **R12** mandated that audio timing stayed aligned with machine execution. Tone.js was considered as an audio backend because it provides convenient musical abstractions, but the final design needed tighter control over the relationship between VM execution, source code locations, and audio scheduling. Implementing an audio engine with the native Web Audio API directly made the audio runtime a consumer of VM execution events rather than a separate scheduler with its own model of musical time.

Chapter 5

Minimum Viable Product

This chapter documents the design, implementation, and evaluation of ASyMphony’s minimum viable product (MVP). An MVP should demonstrate a product’s essential features - enough to gather feedback on the core premise and inform subsequent iterations with real user-tested data. Responses from this user testing are key in guiding the improvements that are made before the final version. The MVP underwent usability heuristic analysis, think-aloud testing, and the System Usability Scale (SUS). It achieved a SUS score of 63.54, indicating below average usability but enough promise to justify a final iteration [3].

5.1 Design

5.1.1 Execution Model

After deciding on a virtual machine model, outlined in Chapter 4, the remaining design focused on **R5**, **R7**, and **R10** in Section 4.2; it needed stateful computation, layered musical parts, and visible execution. A register-based VM was chosen because it made the execution state simple to expose in the interface, while also mapping naturally to musical control. Registers provide a small set of named values that can be inspected and edited during execution. In a conventional machine model, these values might represent counters, intermediate arithmetic values, or addresses. In the MVP, the same idea was repurposed musically: registers could represent parameters such as pitch, duration, volume, or the current position within a pattern. This means that changing a register was both a computational and musical action. Memory provides larger indexed storage for longer-lived values and, importantly, reusable patterns. These could be melodies, rhythmic sequences, or musical parameters that are read over time. This creates a clear connection between musical structure and computer architecture: a melody can be understood as data in memory, while a register can act as an index into that melody. In a typical machine, **output** instructions transfer data from the processor to an external device. In the MVP, virtual audio devices serve an equivalent role: they receive values produced by the program and interpret them as sound. This allows musical output to be presented as a consequence of machine execution, and strengthens the music-machine analogy present throughout the VM’s design.

Most musical performances are layered: a drum pattern, bass line, and melody may progress at the same time while following different rhythms. The VM was therefore given an array of **tracks**, which each hold a compiled instruction sequence, program counter, and musical state. A track represented one sequential musical part. The machine advanced the tracks independently according to their own musical time, creating musical concurrency while keeping each track internally sequential. When the machine reached the end of a track’s instruction stream, the track’s program counter was reset back to zero, which enabled the fundamental ever-looping nature of musical live coding.

5.1.2 Instruction Set

The MVP instruction set was motivated by **R4-R6** (Section 4.2.2) and **R8** (Section 4.2.3): the language needed a small assembly-like vocabulary, each instruction needed a clear purpose, and musical parameters needed to be affected through the same mechanisms as computation. The instruction set borrowed familiar assembly concepts and adapted them to a musical execution model. It was deliberately small, containing only seven opcodes which are shown in Table 5.1. **PLAY** and **REST** produce sound and time; **LOAD**, **STORE**, and **ADD** manipulate machine state; **JUMP** and **JMPZ** provide control flow. It was important

that these keywords were not only easily understandable but also mapped to a concrete and consistent machine action.

Opcode	Operands	Function
PLAY	device, value	Sends a MIDI note value to a virtual musical device for playback.
REST	duration	Suspends the current track for a specified musical duration.
LOAD	register, value	Writes a value into a register.
STORE	address, value	Writes a value into memory.
ADD	register, value	Adds a value to the current contents of a register.
JUMP	label	Sets the program counter to the instruction associated with a label.
JMPZ	register, label	Sets the program counter to a label if the specified register contains zero.

Table 5.1: The opcodes, operands, and functions of instructions present in the MVP.

Syntax	Function
‘identifier’:	Defines a target called ‘identifier’ that jump instructions can skip to.
TRACK ‘identifier’	Defines a new concurrent track called ‘identifier’.

Table 5.2: The syntax and function of additional language constructs.

PLAY acts as the bridge between computation and sound: it takes a target device and a note value, then sends that value to the audio layer. The name is intentionally musical rather than hardware-oriented – OUT would perhaps have been more traditional, but less approachable. REST advances the track’s musical time. Creating silence is as important as creating sound in the construction of musical rhythm.

The machine manipulation instructions explicitly expose registers and memory. LOAD places a value in a register, STORE writes a value to a memory address, and ADD updates a register value in place. ADD is designed as a two-operand instruction, in that it uses the first operand as both a source and a destination. This is less general than a three-operand arithmetic instruction, but is simpler for beginners and supports the common computational and musical cases of incrementing a pitch, address, or counter. The control-flow instructions allow loops and conditional variation, both of which are fundamental to music and programming.

Labels defined jump targets, mirroring typical assembly code syntax. Additionally, the TRACK keyword was introduced as a language-level construct. This separated the source code into concurrent instruction streams to be sent to the VM as **tracks** when compiled. Each TRACK declaration owned subsequent instructions until the next TRACK. Instructions outside of a TRACK declaration belonged to a hidden default **track**. Control instructions within a **track** affected only that **track**’s program counter. These extra language features are listed in Table 5.2.

5.1.3 User Interface

The MVP interface was motivated by **R2** and **R3** in Section 4.2.1, and **R10** and **R12** from Section 4.2.4. Users needed one workspace for code and runtime feedback, guidance, visible machine state, and a basic connection between source code and current sound. Figure 5.1 contains a screenshot of the MVP workspace while executing user code.

The window was split into two halves. The text editor took up the left half of the screen, and register readouts, memory grid, and runtime controls filled the right-hand side. This reflected the central activities of the system: writing code and inspecting execution. A help panel replaced the register and memory view when toggled, allowing users to see their code in the editor at the same time. The editor used line highlights for each track to give users feedback on what their programs were doing at any given moment and make the relationship between the code they write and the sound it generates more apparent, in a similar vein to Gibber and Strudel [24, 26].

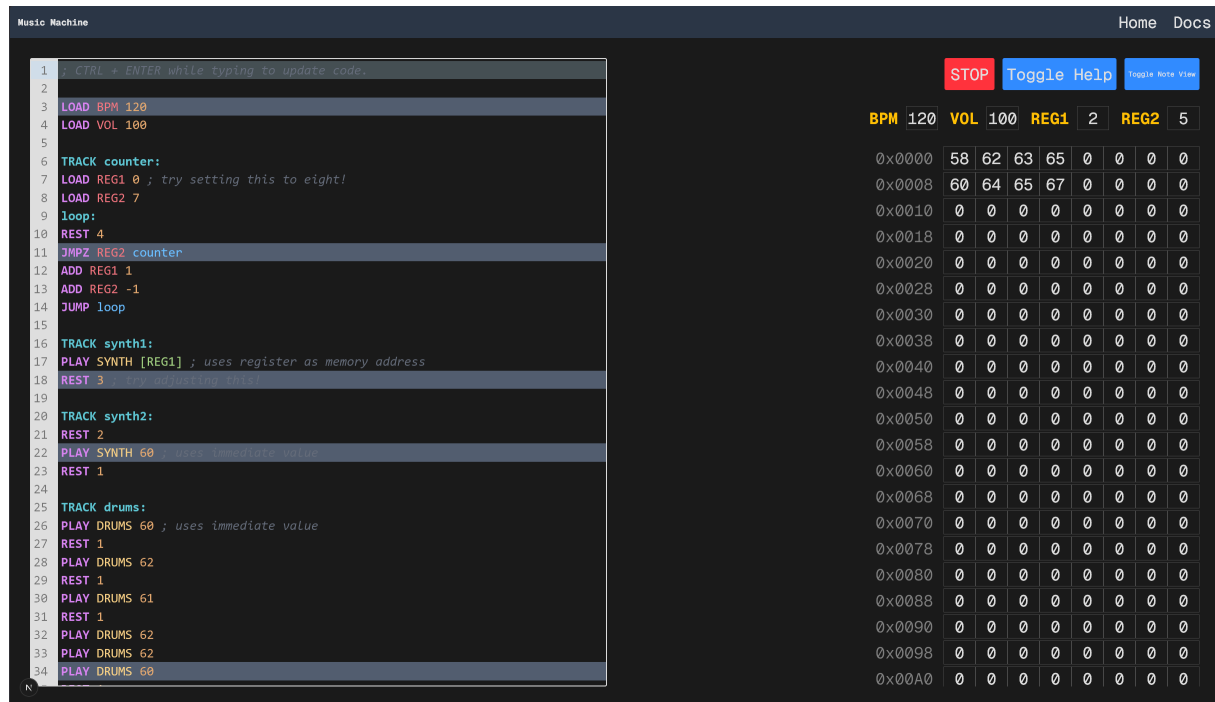


Figure 5.1: Overview of the MVP interface showing code editor with syntax highlighting and line highlights on the left, and controls, registers, and memory on the right.

5.2 Implementation

5.2.1 Audio Engine

The MVP audio engine followed requirements **R7** and **R8** (Section 4.2.3), and **R12** (Section 4.2.4). Sound had to be generated in the browser, respond to machine state, and give immediate feedback during execution.

This led to a primitive audio engine that used the native Web Audio API to initialise an audio context, load drum samples, and trigger samples or synthesised pitches when the machine reached a `PLAY` instruction. The engine exposed two virtual instruments, `DRUM` and `SYNTH`. The `DRUM` device sampled predefined sounds that were mapped to certain MIDI notes. `SYNTH` sounded a single Web Audio oscillator at a frequency derived from the MIDI note operand.

Musical timing was exposed through a `tick` duration derived from current BPM.

The duration of a `PLAY` note was fixed at one tick, while samples played for their entire duration.

5.2.2 Grammar

The MVP language was defined with a Lezer grammar. The grammar recognised tracks, labels, instructions, opcodes, audio devices, registers, immediate values, memory reads, identifiers, and comments. A simplified version of this grammar written in Extended Backus-Naur form in Equation 5.3.

Using a formal Lezer grammar was important even in the MVP. It meant that the system could easily support CodeMirror syntax highlighting with the generated parser, and provided a framework for comprehensive diagnostics later down the line. This would not have been the case with a simple line-splitting parser.

5.2.3 Compiler

The compiler needed to turn compact assembly-like source code into executable tracks while preserving labels and operand categories well enough for editor and runtime feedback.

The compiler had three phases. First, the generated Lezer parser constructed the syntax tree. Secondly, the tree was traversed to identify track declarations, labels, instructions, and operands. Finally, those structures were converted into executable tracks for the virtual machine.

```
program = {statement}
statement = track | instruction | label
track = "TRACK", identifier
instruction = opcode, operand, {operand}
label = identifier, ":"
opcode = "PLAY" | "REST" | "LOAD" | "STORE" | "ADD" | "JUMP" | "JMPZ"
operand = device | register | immediate | address | identifier
device = "DRUMS" | "SYNTH"
register = "BPM" | "VOL" | "REG1" | "REG2"
immediate = [digit], {digit}
address = "[", immediate | register, "]"
identifier = [letter], {letter}
comment = ";", {- newline}
newline = "\n"
```

Table 5.3: ASyMphony’s MVP grammar in Extended Backus-Naur form, where ‘letter’ and ‘digit’ are trivial.

Operands were represented with a small `mode/type/value` structure. Immediate values were parsed as numbers. Registers and devices were implemented as `enums` to ensure type safety. Memory operands were parsed by ignoring the square brackets and checking the whether the address was an immediate value or a register read. Labels were stored as instruction indices, so jumps could target a name instead of a numeric address.

This original compiler was particularly lightweight. It trusted that the source program would be correct and returned tracks directly rather than a success / error result. This made the MVP fast to implement, but it meant that invalid programs failed silently or produced unclear behaviour.

5.2.4 Runtime

The MVP runtime combined the virtual machine, registers, memory, tracks, timing loop, audio engine, and a callback for React state updates. It contained four registers: `BPM`, `VOL`, `REG1`, and `REG2`, as well as a 256-cell memory array. `BPM` and `VOL` controlled global musical parameters, while `REG1` and `REG2` acted as general-purpose registers that could be used for any of a user’s grammatical needs, such as counters, pitches and memory addresses. This split connected computation to music directly. Register updates were not only abstract state changes, but could immediately affect tempo, volume, or note choice.

A JavaScript interval timer was used to schedule `track` execution. The interval was equal to the audio engine’s current `tick` duration (derived from `BPM`). This directly tied execution speed to `BPM`, strengthening the relationship between musical and computational state – changing a musical register had an immediate impact on how the machine runs. One issue that arose was how to deal with non-musical instructions when the `BPM` drives execution. The solution was to continue executing each track until a *time-consuming* instruction was met. Computational instructions were given zero musical time, so as not to interfere with musical structure. `PLAY` instructions were fixed at one `tick` of musical time, and `RESTs` consumed their operand’s worth of time. One limitation with this implementation is that it’s not hard to fabricate a scenario where no time-consuming instructions exist in a `track`. A safety case was added that stopped a `track`’s execution after a certain number of instructions had been executed without consuming musical time.

Loading new code replaced the machine’s `track` list, but attempted to preserve the program counters and musical timing of tracks with the same name. The idea was that users could edit tracks at any time without audible hitches due to restarting from the beginning. This functionality was integral to requirement **R9** (Section 4.2.3), as spontaneous edits are a key part of live coding performance.

5.3 Reflection

The impact of this implementation was that musical feedback was immediate enough for a prototype. The limitation was precision and integration. Audio triggering, source highlighting, and machine-state display were not driven by a single event model. Users could hear notes and see line highlights, but they could not see which memory or register accesses caused the sound. The runtime used JavaScript intervals based on the current `tick` length, while audio playback used the Web Audio clock at the moment of triggering. This meant that the timing of notes drifted over time with micro-hitches of the host system's CPU.

As `PLAY` consumed one `tick` of musical time, it was impossible to construct chords within a single `track`, and interaction between `PLAY` and `REST` had to be considered carefully.

5.4 Evaluation

This section describes the evaluation method used to assess the MVP against the initial design requirements (4.2). The evaluation was performed in two stages. Firstly, the MVP was evaluated against certain usability criteria, detailed in 5.4.1. Secondly, a small user study was performed that consisted of think aloud tasks (5.4.2) and a structured exit survey (5.4.3) for each participant.

5.4.1 Usability Heuristic Analysis

Method

Requirements **R2** and **R3** from Section 4.2.1, and **R10**, **R11**, and **R12** from Section 4.2.4 suggested that ASyMphony had to be easily approachable, understandable, and supportive for beginners. Assessing these characteristics at this stage was useful in determining whether the core design direction was viable, identifying the most significant usability issues, and revealing which aspects of the system were worth preserving in the final iteration.

Jakob Nielsen's Usability Heuristics were used as a standard and structured way to identify usability problems [16]. The MVP was given to an independent evaluator who evaluated it against each of the ten criteria and gave comments, recommendations, and a severity score from 0 to 4 for each issue. A severity of 0 indicates no usability problem, and a severity of 4 indicates a usability catastrophe. This severity score captures three areas: the frequency of an issue, its overall impact on the experience, and how persistent it is [17].

Findings

1. **Comments:** The right-hand grid displaying hexadecimal addresses (e.g. 0x0000, 0x0008, 0x0010) does not clearly communicate what system state it represents, making it difficult for users to understand how it relates to the sequencer. **Recommendations:** Clarify what the grid represents by adding labels or tooltips. Provide visual feedback such as highlighting the current beat, playhead, or active cells so users can easily understand the system's current state. **Severity:** 3
2. **Comments:** The interface uses technical terminology such as register, JMPZ, label, and memory addresses, which may confuse users approaching the tool from a music or sequencing background. **Recommendations:** Provide brief explanations or tooltips to clarify technical terms. Use terminology that aligns more closely with music production concepts where possible. **Severity:** 2
3. **Comments:** Undo and redo actions are supported via keyboard shortcuts but are not visibly indicated in the interface, which may make them harder for users to discover. **Recommendations:** Provide visible UI controls or hints for undo and redo actions. **Severity:** 1
4. **Comments:** The interface introduces novel concepts such as the hexadecimal grid and register-based sequencing that differ from conventions used in most live coding music tools. This may increase the learning curve for users familiar with other environments. **Recommendations:** Align familiar parts of the interface (like the code) with conventions commonly used in live coding tools where possible. Clearly explain the novel concepts so users can understand how they relate to familiar sequencing ideas. **Severity:** 2

5. **Comments:** Code errors or invalid syntax are not highlighted in the editor, which may make it difficult for users to identify mistakes before running the program. Also, refreshing the page resets the workspace, which can result in users losing their work unexpectedly. **Recommendations:** Provide basic syntax validation or visual error highlighting. Consider adding autosave or a warning before refreshing or leaving the page. **Severity:** 4
6. **Comments:** Users must rely on the Help menu to remember command syntax, requiring them to recall information rather than recognizing it. Also, the Help panel cannot be scrolled while audio is playing, which may interrupt the workflow when users want to look at syntax while coding. **Recommendations:** Provide inline syntax hints or autocomplete suggestions in the editor so users can recognize commands without needing to open the Help panel. Allow the Help panel to remain accessible and scrollable while audio is playing. **Severity:** 3
7. **Comments:** A shortcut to comment or uncomment lines of code is not supported. This may slow down users who want to quickly disable or test parts of their code while live coding. Other common editor shortcuts used in coding environments are also not supported. **Recommendations:** Add comment/uncomment line shortcut (Ctrl + /). Consider including other frequently used shortcuts from coding environments (e.g., duplicate line, run current line...). **Severity:** 2
8. **Comments:** Some interface elements (e.g., Help, Start Audio, BPM/volume controls) are visually large relative to the main grid, which may draw attention away from the primary workspace. **Recommendations:** Reduce the size of secondary controls so the main coding and sequencing areas remain the primary focus. **Severity:** 1
9. **Comments:** The interface does not provide error messages when code fails or contains invalid syntax. **Recommendations:** Provide clear error messages that indicate what the issue is and where it occurs in the code. Suggest possible fixes to help users quickly recover from errors. **Severity:** 4
10. **Comments:** The Help section provides basic explanations but is not clearly structured and lacks practical examples, which may make it harder for users to quickly understand how to use the syntax. **Recommendations:** Organize the Help content more clearly using structured sections or lists. Include short code examples to demonstrate how commands are used in practice. **Severity:** 3

Implications

The heuristic evaluation indicates that, although the MVP represented the main concepts of the project well, it fell short on the onboarding and transparency requirements. The interface did not yet provide guidance for new users, mistake feedback was particularly weak, and the lack of keyboard controls offered little flexibility for advanced users. The most important lesson was that exposing machine state alone was not sufficient to guarantee user understanding. ASyMphony’s educational value depends on user being able to connect code, state, and sound; visibility had to be supported by stronger feedback and better guidance for first-time users. These findings are valuable as they clarify which areas in particular need prioritising for the next development phase.

5.4.2 Think-Aloud Sessions

Method

Think-aloud testing asks participants to verbalise their thoughts while using a system [18]. It is a quick and easy way to get an insight into users’ expectations of the system, whether it meets them or not, and in which areas. For ASyMphony’s think-aloud sessions, example code was loaded into the editor and participants were asked to explore the interface, start and stop playback, and modify values to hear the result. 12 participants attended the study, most of whom were computer science students, with the rest from non-computing backgrounds. None were selected especially for their musical background.

Findings

Participants responded positively to the immediate audible feedback. They found the **PLAY** and **REST** instructions intuitive, liked being able to mute or comment out sounds, and praised the responsiveness of playback and line highlights in the source code. Several participants found the example comments helpful because they made the editable parts of the program less intimidating.

The main problems were about understanding the current state and available actions. Users were often unsure when code changes had been applied. Some expected familiar save shortcuts such as **Ctrl+S** rather than **Ctrl+Enter**. Several participants did not realise that tracks were running in parallel, or found the **TRACK** structure confusing because parallel execution was hidden inside one source file. Memory addressing was also unclear – users wanted more obvious address labels, highlighting for accessed memory, and a clearer explanation of indirect access. The need for error feedback, a guided tutorial and more detailed help information was a repeated criticism.

Implications

The think-aloud sessions confirmed the same pattern as the heuristic analysis. Users like the premise and immediate feedback, but the MVP did not yet explain its own workings. Help documentation, a comprehensive onboarding system, clear error feedback, and an overhauled concurrent execution system were key areas that needed improvement in the final product.

5.4.3 Exit Survey

Method

After using the MVP, participants completed the System Usability Scale (SUS), a standard ten-item questionnaire that serves as a compact measure of perceived usability [4]. SUS was useful because it produces a compact score from 0 to 100 and can be compared across iterations.

Each item is answered on a five-point scale. Odd-numbered items are framed positively, so their contribution is the response minus one. Even-numbered items are framed negatively, so their contribution is five minus the response. The sum is multiplied by 2.5 to produce a score from 0 to 100:

$$\text{SUS} = 2.5 \times \left(\sum_{\text{odd}} (q_i - 1) + \sum_{\text{even}} (5 - q_i) \right)$$

Results

The recorded SUS scores had a mean of 63.54 across the 12 responses. This is below Bangor et al.’s published ‘good’ adjective band for SUS scores, indicating that the MVP was usable enough to test, but not polished enough to be a strong standalone tool [3].

Implications

The SUS results align with the qualitative evidence. Users could see the value of the system and were able to interact with it productively, but encountered enough friction that perceived usability suffered.

5.5 Summary

The design implication was clear: the final product needed to make the machine model more visible and explicit. The strongest areas to preserve were the minimal browser-based interface, the small and readable language, immediate audible and visible feedback, and the integration of music and machine. The weakest areas that needed implementing were the onboarding experience, code diagnostics, visible parallelism, register and memory access visuals, and richer musical control. Table 5.4 summarises the MVP against the full requirements set.

Table 5.4: MVP evaluation against the implementation requirements.

Req.	Status	Evidence
R1	Met	The MVP ran as a browser application and did not require specialist music or compiler software.
R2	Partially met	The editor, execution controls, audio, registers, and memory were in one workspace, but the help panel replaced the state view and the interface did not yet feel fully integrated.
R3	Weak	The MVP included a help panel and examples, but onboarding was thin and users still struggled to understand what to do first.
R4	Met	The seven-opcode syntax was compact and memorable enough for a prototype.
R5	Partially met	Registers, memory operands, labels, loops, and conditional jumps existed, but invalid operand handling was missing.
R6	Met	Each opcode had a clear role: sound, timing, state manipulation, or control flow.
R7	Partially met	TRACK supported layered musical parts, but parallel execution was hidden inside one editor.
R8	Partially met	BPM, VOL, registers, and memory could affect sound, but panning, envelopes, timbre, and sample mapping were limited.
R9	Partially met	Users could edit code and reload tracks during use, but the load/run relationship was not clear enough.
R10	Partially met	Registers, memory, line highlights, and running status were visible, but errors and fault states were weak.
R11	Weak	Parallel execution existed conceptually, but users could not see independent execution contexts clearly.
R12	Weak	Line highlighting connected code to playback, but reads, writes, source highlights, and audio events were not driven by one event model.

Chapter 6

Final Product

This chapter documents the design, implementation, and evaluation of ASyMphony’s final product. The final design was driven by the findings of the MVP. Users liked the central idea of the project, but needed clearer concurrency, better onboarding, stronger diagnostics, and richer musical control. This is reflected in the design requirements that the MVP only partially met. **R5**, **R7**, **R10**, **R11**, and **12** (Section 4.2) dictated more exposed execution feedback, explicit simultaneous parts, and a stronger connection between code, state, and sound. Evaluation methods for the final product mirrored the MVP so that results were directly comparable. Heuristic evaluation found far fewer and less severe usability issues. The mean SUS score across twelve participants was 81.04, an improvement of 17.50. This places the final product in Bangor et al.’s [3] ‘good’ usability band.

6.1 Design

6.1.1 Multi-Core Separation

Separating the execution model into multiple cores was motivated by **R7**, **R10**, and **R11** from Section 4.2). The system needed multiple simultaneous parts, visible execution state, and an explicit representation of parallel execution. The final product contains four cores, each with their own code editor, loaded program, program counter, musical timing, and register file. All cores share a 32-cell memory array and two global parameters, BPM and master VOL.

The number of cores was a design choice to allow for common musical layering such as drums, bass, melody, and chords, while still fitting comfortably on a desktop screen. This directly addresses the MVP confusion around **TRACK**, as parallelism was made spatially apparent with separate execution areas in the interface rather than hidden in one text editor.

6.1.2 Timing and Transport

The MVP tied execution speed directly to the global BPM. Tracks executed until reaching a time-consuming instruction such as **PLAY** or **REST**. This made simple playback easy, but limited polyphony and made rhythmic reasoning harder. The final product separates machine execution from audio scheduling with a musical transport model. The transport converts between musical **ticks** and browser audio time using the current BPM, and each core keeps track of its own **tick** position.

This design allows **PLAY** to schedule a note without advancing a core’s musical time. **REST** is now the only way to advance a core’s time. **PLAY** now also can take an optional duration operand. This makes duration a musical parameter rather than a fixed value.

6.1.3 Registers and Memory, and Instruction Set

The final register file is larger and more musically integrated than the MVP. Each core has:

VOL	:	per-core volume
PAN	:	stereo panning
ATK	:	note attack time in milliseconds
REL	:	note release time in milliseconds
REG0–REG3	:	general-purpose registers
RAND	:	random-number bound; reads produce a random value

This design makes musical parameters part of the same machine-state model as the rest of computation. A user can write `LOAD PAN -30`, `ADD REG0 1`, or `LOAD VOL 40` using the same instruction form. `RAND` support musical variation while following the same model as other musical parameters.

Shared memory remains important for holding joint melodies and rhythms. Memory is kept small, at only 32 addresses. This is easier to display and inspect at once, which improves transparency and allows room for the explicit core design.

The final product retains the compact seven-opcode ISA of the MVP (Table 5.1) because users found it to be clear and understandable. `PLAY` is extended with an optional note duration to allow for increased musical control without introducing extra instructions.

Operands are defined as one of five types: immediate values, registers, addresses, devices, and labels. Immediate values are number literals. Register operands name per-core registers. Address operands wrap a value in square brackets, such as `[12]` or `[REG0]`. Device are one of `SYNTH`, `DRUMS`, `BASS`, or `PIANO`. Label operands name program positions. This typing allows for stricter error feedback and lets invalid programs be rejected before reaching the runtime.

6.1.4 Error Feedback

The final error model was driven by feedback from MVP evaluation and requirements **R10** and **R12** in Section 4.2.4. Errors need to be visible, and users need to be able to easily connect failures to their cause.

The compiler’s responsibility is therefore expanded to validate the number of operands and their types for each encountered opcode. Diagnostics from this process are shown in the code editor with accurate source code positioning. This lets users know exactly which token has caused the error and shortens the path to fixing it. Figure 6.1 shows this in action.

Runtime faults are also made visible rather than just failing silently. Invalid note durations or memory accesses by a specific core can stop its execution and show the reason for the fault to the user. Figure 6.2 shows an example. This is important because not all problems are static. A memory address held in a register might only become invalid during execution, for example if incremented past the memory size. By separating compile-time diagnostics from runtime faults, ASyMphony gives users a more accurate failure model which can aid in understanding what went wrong and why.



Figure 6.1: View of one core showing accurate syntax error diagnostics.

6.1.5 Tutorial System and Sample Map

Feedback from the MVP made it clear that a tutorial system was required to increase accessibility and accomplish requirement **R3** (Section 4.2.1). Instead of relying only on a static help panel, the final product presents staged lessons. The lessons introduce the interface, programs, cores, master controls, instruction syntax, registers, memory, and jumps. Each lesson contains short explanations of concepts, syntax examples, and a final practical task.

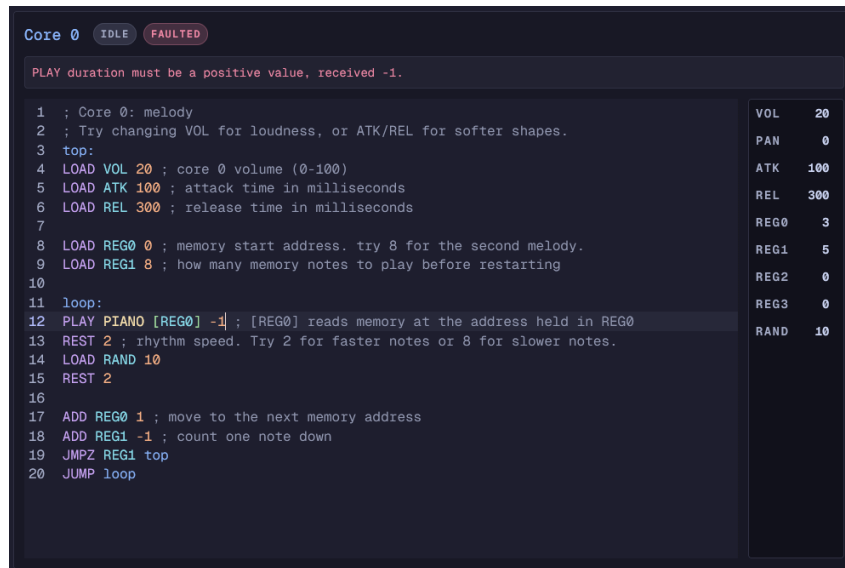


Figure 6.2: View of one core showing the result of a runtime fault.

This tutorial system is important because ASyMphony introduces unfamiliar concepts even to users who have high-level programming experience. It breaks these ideas into small tasks and uses the interface itself to direct users’ attention and understanding.

Figure 6.3 shows the ‘task’ step of the first lesson, which requires users to replace placeholders in the code with immediate values, before loading and running the program.

The final product also includes a sample mapper for selecting drum sounds. Users can assign different samples to MIDI note values and remove or replace existing mappings. This improves musical expressiveness (R8 in Section 4.2.3) without overhauling the instruction set. The same `PLAY DRUMS note` instruction can produce different sounds depending on the sample mapped to that `note`.

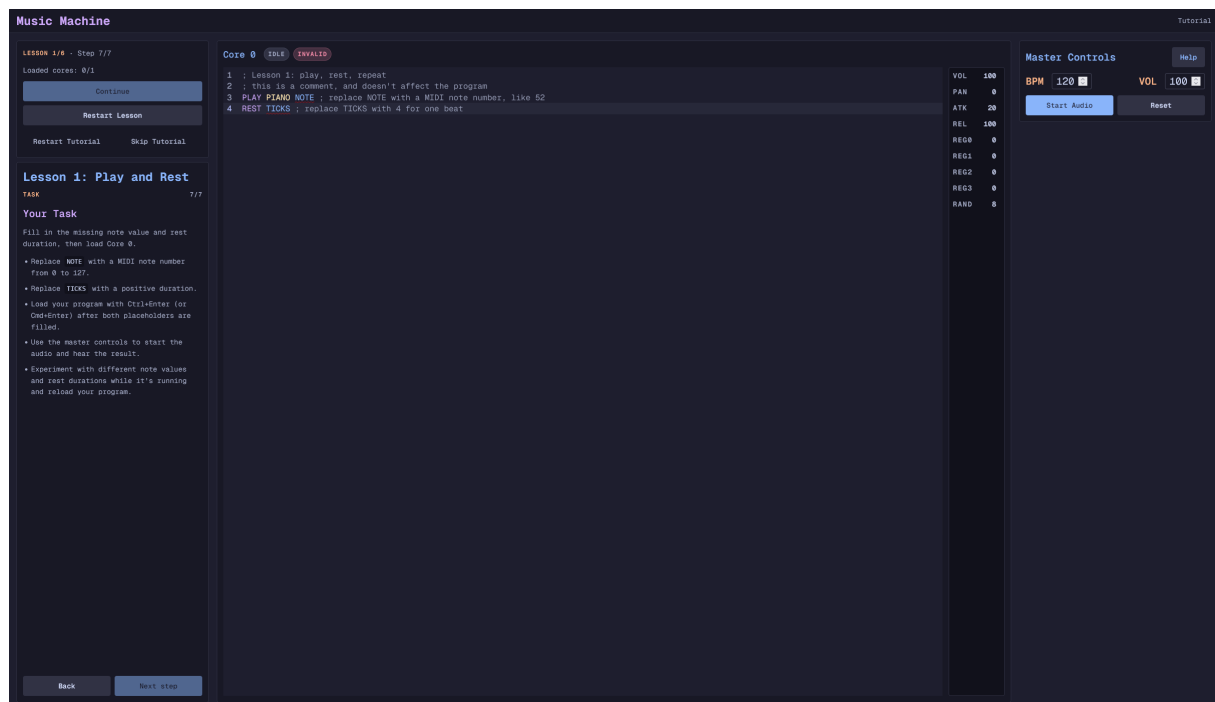


Figure 6.3: Screenshot of the tutorial interface showing the ‘task’ step of the first lesson.

6.2 Implementation

6.2.1 Runtime Architecture

The final implementation split the MVP’s runtime into clearer modules: a runtime manager, CPU, cores, register files, shared memory, musical transport, highlight manager, and audio engine. Figure ?? shows this architecture. The separation matters because each module had a more distinct task. This made it easier to develop and expand the modules as new features were required.

6.2.2 Compiler and Diagnostics

The final compiler is a significant improvement over the MVP’s implementation. Its responsibilities have expanded from extracting instructions to validating the user’s program before it reaches the runtime. The compiler returns a `CompileResult` containing either a typed program or a list of diagnostics. This is important for usability – invalid code is prevented from reaching a core, and users are shown what went wrong and where.

The front-end of the compiler remains the same. Lezer’s generated parser takes source code and attempts to construct a syntax tree. This time though, Lezer’s diagnostics are given to CodeMirror using the linter extension, showing syntax errors directly from the parse tree in-editor. If parsing succeeds, the compiler then builds a raw representation of instructions and labels. Labels are collected into a table of instruction addresses, with duplicates reported as diagnostics. Each instruction is then converted according to the expected operand types for its opcode. This enforces the ISA outlined in Table 5.1 before it reaches the runtime.

This design improves upon **R10** and **R12** (Section 4.2.4) because every diagnostic holds information about the section of code that caused it, and this can be used to clearly show which code section needs changing to the user.

6.2.3 CPU, Cores, and Memory

The final runtime separates the machine into a CPU, four `Core` objects, a shared `Memory` object, and shared global parameters. The CPU owns the memory and global parameters, while each core owns its loaded program, program counter, musical tick, and register file.

Each core executes one typed instruction at a time. State manipulation instructions update registers or memory; control-flow instructions modify the local program counter; **PLAY** creates a note event; and **REST** advances the core’s musical `tick`. Memory updates are checked for out-of-bounds access, which would raise a runtime fault in the offending core and halt its execution. Invalid **PLAY** or **REST** durations are handled similarly.

6.2.4 Highlights

Each execution step of a core produced an `ExecutionEvent` that contained any notes to be scheduled, the source code region, which registers or regions of memory were modified, and which tick it happened at. A highlight manager used this execution information to schedule appropriate highlights of source code and registers or memory.

6.2.5 Scheduling

The runtime uses a lookahead scheduler to advance each core [5]. Using a 12ms JavaScript interval, it asks the transport for a horizon `tick` that is 0.2 seconds in the future. The CPU then executes all cores until they are all past the tick horizon.

6.2.6 Tutorial

The tutorial system was implemented as a separate page using Next.js’s page routing. Users were redirected there if it was their first time using the app. It used the same layout as the usual workspace, but with an added `lesson` panel on the left-hand side, containing tutorial and sub-lesson controls. There were six lessons, each with ~3-6 steps.

Lessons were created by filling an array of `CodeLesson` structs. The panel used this array to step through the lessons. The steps spotlighted relevant UI components, explained relevant concepts and syntax, and gave users a gated task that had to be completed before they could move on.

6.3 Reflection

The final implementation addresses the MVP’s main weaknesses. Parallelism is explicit, errors are visible, and audio scheduling is more precise. This adds more complexity to the system, but the tutorial system introduces these new ideas in a progressive and structured way in hopes of mitigating any increased conceptual friction. A remaining limitation is that the user interface is desktop-oriented, and little thought was put into a responsive or mobile-friendly option. This limits the accessibility of the system, and proper small-screen support could be pursued as future work.

Figure 6.4 shows the final system in its running state.

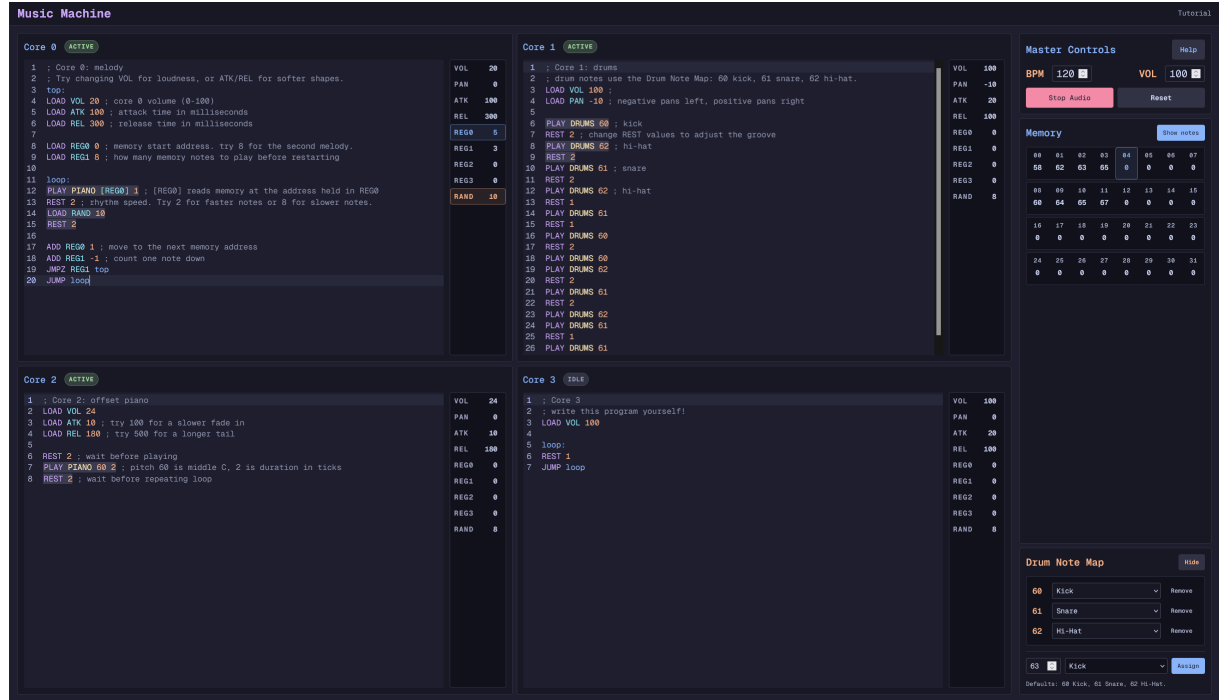


Figure 6.4: Final interface in its running state, showing four cores, per-core registers, global controls, shared memory, the drum note map, register and memory highlighting and source code highlights.

6.4 Evaluation

The final product was evaluated with the same heuristic approach and SUS survey as the MVP, allowing for direct comparison between iterations. Twelve new participants used the final product, interacted with the guided tutorial, then completed an exit survey containing the SUS and twelve additional Likert-scale items [12]. These items focused on participants’ engagement and perceived learning of assembly concepts after using the system, and provide evidence for the evaluation of objectives **O1-O5**.

6.4.1 Usability Heuristic Analysis

Method

Nielsen’s usability heuristics again provided a structured source of usability feedback [16]. The final product was given to a different evaluator, who recorded comments, recommendations, and severity ratings for each usability heuristic. The same severity rating was used, considering the frequency, impact, and persistence of issues, where 0 indicates no usability problem and 4 indicates a usability catastrophe [17].

Findings

1. **Comments:** Good highlighting of current position / status throughout. **Severity:** 0.

2. **Comments:** Core concept is to model CPU/Assembly rather than real world, but the syntax and UI are understandable / relatable. **Severity:** 0.
3. **Comments:** Undo / redo key commands are available for code editors (and other text inputs such as memory registers) but they only apply to the current text input, rather than being a global undo history. **Recommendations:** Implement a global undo history. **Severity:** 1.
4. **Comments:** Could have incorporated commonly used icons for some actions e.g. play >, stop □, remove (bin), show +, hide −, help ? - but that might have created too much visual clutter. For keyboard users, it's not clear how to exit a code window; the tab key only adds tabs into the editor. **Recommendations:** Perhaps disable tab input in the editor so it can be used for navigation instead, or add shortcut key navigation e.g. alt-1 to set focus to Core 1, alt-m to focus on Master Controls (although might clash with browser shortcuts). **Severity:** 1.
5. **Comments:** Auto-complete, invalid status and error highlighting work well. Code data saves automatically, but not memory values. **Severity:** 0.
6. **Comments:** Undo / redo key commands explained in the tutorial and help, perhaps having buttons as well would be more obvious. Similarly having a button action as well as ctrl-enter to apply / load changes. **Recommendations:** Perhaps add a button on each core for the ctrl-enter action, or use the “unloaded” button. **Severity:** 1.
7. **Comments:** Ability to quickly comment out lines is useful. Ability to save and reload programs e.g. to disk would give more flexibility. Running different programs in different browser tabs doesn't work; they seem to share the same data. **Recommendations:** Change auto-save mechanism to save data per browser tab. Implement save/load from disk. **Severity:** 1.
8. **Comments:** Good design, but doesn't render well on smaller screened devices. **Severity:** 1.
9. **Comments:** Auto-complete, invalid status and error highlighting work well. **Severity:** 0.
10. **Comments:** Very good help section and tutorial. **Severity:** 0.

MVP Comparison

The heuristic analysis supports the claim that the final product addressed the MVP's most important usability weaknesses. The final interface is more complex, but the complexity is explained and structured. The remaining issues are real, especially keyboard navigation and small-screen layout, but they do not undermine the core usability of the system.

Table 6.1 displays the change in observed severity from the MVP for each heuristic. The severity for each heuristic always either reduced or remained the same, often being fully eliminated. Some issues remain, but the number of issues drastically decreased, suggesting that the project is much more supportive and approachable for beginners.

Table 6.1: Comparison of issue severity for each usability heuristic between the MVP and final product.

Heuristic	MVP severity	Final severity	Change
Visibility of system status	3	0	-3
Match between system and real world	2	0	-2
User control and freedom	1	1	0
Consistency and standards	2	1	-1
Error prevention	4	0	-4
Recognition rather than recall	3	1	-2
Flexibility and efficiency of use	2	1	-1
Aesthetic and minimalist design	1	1	0
Recognise, diagnose, and recover from errors	4	0	-4
Help and documentation	3	0	-3

6.4.2 System Usability Scale

Method

Participants used the guided final product and then completed SUS. The new scores were compared against the MVP’s results.

Findings

The final product achieved a mean SUS score of 81.04, compared with the MVP mean of 63.54. This is an improvement of 17.50 points. Bangor et al. [3] note that SUS scores around the low 80s are commonly associated with strong perceived usability. The result therefore supports the claim that the final product is perceived highly usable.

MVP Comparison

The SUS result is important because the final product is visually and conceptually more complex than the MVP. It includes four separate cores, more registers, more interface panels, a tutorial system, code diagnostics, and a drum sample mapper. The score suggests that this added complexity did not harm usability overall. Instead, the added structure and onboarding made the system easier to use.

6.4.3 Questionnaire

Method

The final evaluation included twelve additional five-point Likert-scale items after the SUS [12]. Six measured user engagement and the impact of audio integration, and six measured perceived learning and confidence.

Findings

Open comments support the numerical pattern. Participants repeatedly described the system as engaging, polished, easy to follow, and helped by the tutorial. Several also identified remaining conceptual difficulty around registers and jumps. This is valuable because it matches the quantitative split – engagement and confidence were strong, while perceived understanding of specific low-level concepts was more reserved.

Table 6.2 displays the mean and standard deviation of the responses.

Table 6.2: Final questionnaire results, scored from 1 (strongly disagree) to 5 (strongly agree). The audio-distraction item is negatively framed.

Questionnaire item	Mean	SD
Tutorial introduced the required concepts.	4.58	0.67
The experience was engaging.	4.75	0.45
Wanted to keep experimenting.	4.42	0.67
Live musical output made programming more interesting.	4.67	0.65
Audio feedback was distracting rather than helpful.	1.25	0.45
Audio and visual output helped show what the program was doing.	4.58	0.51
Better understanding of instruction-based programming.	3.83	1.19
Better understanding of registers.	3.58	1.24
Better understanding of memory reads and writes.	3.83	1.19
Better understanding of control-flow instructions.	3.67	1.07
Confident modifying or writing assembly-like code.	4.42	0.51
Useful introduction to assembly or computer architecture concepts.	4.50	0.67

Implications

ASyMphony appears to make assembly-style programming engaging and approachable. Users strongly valued the live musical output, tutorial, and combined audio-visual feedback.

However, there is still a conceptual gap. The results suggest that ASyMphony is a promising introduction to assembly programming, but users were slightly less confident in their greater understanding of assembly concepts.

6.4.4 Summary

The final product achieved the main usability and engagement goals of the dissertation. It improved SUS compared to the MVP by 17.50 points, received only low-severity heuristic issues, and was rated strongly for engagement, tutorial quality, audio-visual feedback, confidence, and introductory usefulness.

Table 6.3 compares the final product against the full set of requirements. This comparison demonstrates why the final product is stronger as a usable and engaging tool than the MVP. The gaps that remained around user onboarding and diagnostic feedback have been closed by making them key focuses of the final design.

Table 6.3: Final product evaluation against the implementation requirements.

Req.	Status	Evidence
R1	Met	The final product runs in the browser and preserves the low-friction access demonstrated by the MVP.
R2	Met	Code editors, core controls, master controls, memory, audio output, the tutorial system, and the drum sample mapper are all integrated into one workspace.
R3	Met	Guided lessons, examples, help text, and starter code give new users a way in to editing and running programs.
R4	Met	The final language keeps the seven-opcode ISA but now enforces operand structure through compiler checks.
R5	Met	Registers, shared memory, direct and indirect memory operands, labels, loops, and conditional jumps support transferable assembly thinking.
R6	Met	Each opcode has a distinct role in music construction, state manipulation, or control flow.
R7	Met	Four cores support simultaneous musical parts without hiding them inside one code editor.
R8	Met	VOL, PAN, ATK, REL, device operands, duration operands, and sample mappings shape sounds using the machine analogy.
R9	Met	Users can edit code while playback is running and deliberately reload a core with Ctrl+Enter or the interface controls.
R10	Met	Registers, memory, program position, playback status, compiler diagnostics, and runtime faults are visible.
R11	Met	Concurrency is explicit through four visible cores, each with its own editor, registers, status, and program counter.
R12	Met	Source highlights, register access highlights, memory access highlights, scheduled audio events, and runtime faults are all derived from machine execution events.

Chapter 7

Critical Evaluation

This chapter provides a critical evaluation of the project as a whole. An initial overview discusses the extent to which ASyMphony makes assembly programming more engaging and usable through live coding. Limitations of the methods of evaluation are then explored, followed by an evaluation against the five objectives outlined in the introduction.

7.1 Overview

ASyMphony is positioned best as a proof of concept for approachable, engaging assembly-style programming. The final product demonstrates that exposed machine state and live musical feedback can be combined in a usable browser interface.

The evidence is most compelling for its usability and engagement. The final product improved substantially over the MVP in both heuristic feedback and SUS score, moving from 63.54 to 81.04. This suggests that the extra visual complexity introduced by the four-core design did not make the system harder to use overall. Instead, better onboarding, clearer feedback, and more explicit state made the final interface easier to understand than the simpler MVP. Users found the final product to be considerably engaging and that the audio and visual output supported the assembly programming tasks well.

The evidence is weaker for educational viability and musical expressiveness. Qualitative user feedback supports the claim that users could make useful connections between code, state, and sound. However, the project did not measure objective learning gain, long-term retention, or the transfer to conventional assembly tasks. It also did not directly evaluate whether users could create musically satisfying pieces after extended use with the system. ASyMphony makes a compelling argument that assembly programming can be made more usable and engaging through musical live coding, but further work is required to prove educational impact or performance value.

7.2 Limitations

7.2.1 Heuristic Analysis

Both heuristic analyses used a single evaluator. Nielsen’s severity-rating guidance argues that ratings from a single evaluator are unreliable, while mean severity ratings become more reliable when several evaluators rate the same problems [17]. The heuristic results are still useful, but incomplete. They identify valid design issues and show improvement between iterations, but should not be taken as an exhaustive list of usability problems.

The MVP and final heuristic analyses were also conducted by different evaluators. This limits the direct comparison that can be made between their findings. The final product’s low severity ratings are promising, but could be attributed to differences in the individual evaluator. Stronger evidence comes from the SUS improvement and participant comments.

7.2.2 Questionnaire

The final questionnaire measured participants’ self-reported experience. This is appropriate for engagement and perceived usability, but limits the conclusions that can be made about the educational value

of ASyMphony. A stronger evaluation would include pre- and post-test assessments of users' assembly programming understanding.

Participants in the user studies were primarily computer science students or technically confident users. This may have inflated the perceived confidence results compared with a broader beginner population. Additionally, users with musical backgrounds but little programming experience may encounter different issues. Data about participants' programming and musical abilities were not collected during the study, and would have added depth to the conclusions that can be made about beginner accessibility.

7.2.3 Musical Expression

ASyMphony supports creating musical patterns, musical layering, sample selection, random variation, and controlling pitch, duration, volume, panning, attack, and release. However, the extent of its musical expressiveness was not directly evaluated. Participants responded positively to musical output making programming more interesting, but they were not asked to create complete musical pieces over an extended session.

ASyMphony's instruction set is intentionally small, which helps beginners but may restrict experts in musical expression. Musical live coding systems are primarily performance tools that allow for the creation of complex musical experiences. If ASyMphony is to compete with mature live coding environments as a performance language, the extent of its musical expressiveness should be tested.

7.2.4 Design Trade-Offs

The final product deliberately increases visual complexity compared with the MVP by exposing separate cores and more diagnostic and runtime feedback to users. The final SUS score suggests that this decision was successful on desktop, but heuristic evaluation identified smaller screens as a weakness. ASyMphony is therefore best utilised as a desktop browser tool. This limits the reach of ASyMphony, but can be justified by the screen space required to visualise its complex workings and the prevalence of large-screened desktops and laptops in educational environments. Increasing usability on mobile phones or tablets would require a redesigned layout rather than minor styling adjustments.

The language also makes compromises. ASyMphony's compact ISA supports learnability, but limits expressive range. A special `RAND` register is useful for musical variation, but is not a conventional assembly concept. Per-core sound-shaping registers make musical control efficient, but again stray from architecture simulation conventions. Comparisons with established educational tools like the LMC paradigm should be aware that ASyMphony aligns more with a live-coding musical instrument than a strict assembly simulator.

7.3 Evaluation Against Objectives

7.3.1 O1: Accessibility

O1 was achieved well. The final product runs in the browser, combines a code editor, runtime, visualisations, a comprehensive tutorial, and audio output in one workspace, and avoids the installation hurdles associated with many live coding systems. Users arrive directly in a programming environment and can begin listening to programmed music immediately. There are still limitations to the accessibility of the final product. The interface is best suited to desktop screens, and the evaluations did not widely assess accessibility for non-programmers or users with accessibility needs.

7.3.2 O2: Conceptual Transparency

O2 was partially supported by evaluation. The final product makes program counters, registers, memory, compile-time diagnostics, runtime faults, and memory access explicitly visible. This is a significant improvement over the MVP, where machine state was made visible but not strongly connected to execution. Questionnaire responses suggest that users saw the environment as a useful introduction to architecture concepts, but they were less confident in their perceived learning than their engagement with the system. The evidence therefore supports perceived conceptual transparency rather than a proven increase in users' understanding of assembly programming ideas.

7.3.3 O3: Musical Engagement

O3 was well supported. The best results from the final questionnaire came from the engagement, live musical output, and audio-visual feedback questions. Participants did not generally find the audio distracting, and open comments repeatedly described the system as interesting and enjoyable to use. This supports the claim that music can make assembly-style programming more approachable and compelling.

7.3.4 O4: Musical Expressiveness

O4 could not be supported by evaluation due to the methods of assessment. Participants were not asked to create complete musical pieces over an extended session, and therefore they had no opportunity to develop complex or highly expressive pieces. Musical expression was thus not directly evaluated.

7.3.5 O5: Educational Viability

O5 was largely supported by evaluation. The results from the questionnaire show that ASyMphony provides a useful introduction to assembly concepts, and that the tutorial was a helpful educational tool. However, the method of assessment was limited in that it was unable to fully gauge users' understanding of assembly concepts. While participants were able to self-report their levels of confidence, this was not a precise measure of increase in knowledge. A pre- and post-test assessment of users' understanding of assembly programming would better evaluate the programme's educational value.

Chapter 8

Conclusion

8.1 Summary

This dissertation documents the design, implementation, and evaluation process of ASyMphony, a musical live coding environment for the introduction of assembly programming.

ASyMphony achieves the central aim of this project by making assembly programming approachable and engaging with a live coding environment. In particular, it demonstrates an accessible (**O1**), conceptually transparent (**O2**), and musically engaging (**O3**) solution.

It exists as a self-sufficient educational tool and live coding environment for assembly programming. It is hosted on GitHub Pages so is accessible to anyone with a web browser and an internet connection. The project's source code is also made available on GitHub.

8.2 Contributions

- An assembly-like language that supports arithmetic, state modification, audio playback, looping.
- A compiler that takes source code in this language and creates an executable program.
- A multi-core virtual machine that executes these compiled programs.
- A user interface that exposes the workings of this virtual machine to increase conceptual transparency with low-level systems.
- A browser audio engine capable of producing synthesised and sampled sounds using the Web Audio API.

8.3 Future Works

- Evaluate the concrete educational capacity of ASyMphony with pre- and post-test assessments of participant's assembly programming skills and concepts.
- Assess how complex the musical pieces that are created with ASyMphony can be. This could include a comparison to the capabilities of other live coding environments, or conducting further user studies with professional musicians.
- Extend the environment with quality-of-life features like supporting code downloads and uploads, or adding a place where users can share their creations.
- Redesign the interface to be compatible with smaller-screened devices to cater to a wider population.

Bibliography

- [1] Sam Aaron, Alan F. Blackwell, and Pamela Burnard. The development of Sonic Pi and its use in educational partnerships: Co-creating pedagogies for learning computer programming. *Journal of Music, Technology & Education*, 9(1):75–94, 2016. doi:10.1386/jmte.9.1.75_1.
- [2] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2 edition, 2006.
- [3] Aaron Bangor, Philip T. Kortum, and James T. Miller. Determining what individual SUS scores mean: Adding an adjective rating scale. *Journal of Usability Studies*, 4(3):114–123, 2009. URL: <https://uxpajournal.org/determining-what-individual-sus-scores-mean-adding-an-adjective-rating-scale/>.
- [4] John Brooke. SUS: A quick and dirty usability scale. In Patrick W. Jordan, Bruce Thomas, Bernard A. Weerdmeester, and Ian L. McClelland, editors, *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [5] Chris Wilson. A tale of two clocks, 2026. Accessed: 2026-05-03. URL: <https://web.dev/articles/audio-scheduling>.
- [6] Chuck Team. ChucK Language Specification: Time, 2026. Accessed: 2026-05-03. URL: <https://chuck.stanford.edu/doc/language/time.html>.
- [7] Chuck Team. WebChucK IDE, 2026. Accessed: 2026-05-03. URL: <https://chuck.stanford.edu/ide>.
- [8] CodeMirror. CodeMirror system guide, 2026. Accessed: 2026-04-21. URL: <https://codemirror.net/docs/guide/>.
- [9] Gibber Team. Gibber playground, 2026. Accessed: 2026-05-03. URL: <https://gibber.cc/playground/index.html>.
- [10] Hundred Rabbits. ORCA: Esoteric programming language, 2026. Accessed: 2026-05-03. URL: <https://github.com/hundredrabbits/Orca>.
- [11] Lezer. Lezer system guide, 2026. Accessed: 2026-04-21. URL: <https://lezer.codemirror.net/docs/guide/>.
- [12] Rensis Likert. A technique for the measurement of attitudes. *Archives of Psychology*, 22(140):1–55, 1932.
- [13] Sean McCoid, Jason Freeman, and Brian Magerko. EarSketch: An integrated approach to teaching introductory computer music. *Organised Sound*, 18(2):146–160, 2013. doi:10.1017/S1355771813000104.
- [14] Alex McLean. Making programming languages to dance to: live coding with tidal. In *Proceedings of the 2nd ACM SIGPLAN International Workshop on Functional Art, Music, Modeling & Design, FARM '14*, page 63–70, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2633638.2633647.
- [15] Next.js. Next.js documentation: Static exports, 2026. Accessed: 2026-04-21. URL: <https://nextjs.org/docs/pages/guides/static-exports>.

- [16] Jakob Nielsen. 10 usability heuristics for user interface design, 1994. Accessed: 2026-04-30. URL: <https://www.nngroup.com/articles/ten-usability-heuristics/>.
- [17] Jakob Nielsen. Severity ratings for usability problems, 1994. Accessed: 2026-04-30. URL: <https://www.nngroup.com/articles/how-to-rate-the-severity-of-usability-problems/>.
- [18] Jakob Nielsen. Thinking aloud: The #1 usability tool, 2012. Accessed: 2026-04-30. URL: <https://www.nngroup.com/articles/thinking-aloud-the-1-usability-tool/>.
- [19] David A. Patterson and John L. Hennessy. *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. Morgan Kaufmann, 2 edition, 2020.
- [20] React. React documentation: Describing the UI, 2026. Accessed: 2026-04-21. URL: <https://react.dev/learn/describing-the-ui>.
- [21] Replit. Ace, CodeMirror, and Monaco: A Comparison of the Code Editors You Use in the Browser, 2021. Accessed: 2026-04-30. URL: <https://blog.replit.com/code-editors>.
- [22] Mitchel Resnick, John Maloney, Andrés Monroy-Hernández, Natalie Rusk, Evelyn Eastmond, Karen Brennan, Amon Millner, Eric Rosenbaum, Jay Silver, Brian Silverman, and Yasmin Kafai. Scratch: Programming for all. *Communications of the ACM*, 52(11):60–67, 2009. doi:10.1145/1592761.1592779.
- [23] RISC-V International. The RISC-V instruction set manual, volume i: Unprivileged architecture, 2025. Ratified version 20250508. URL: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf.
- [24] Charles Roberts and JoAnn Kuchera-Morin. Gibber: Live coding audio in the browser. In *Proceedings of the International Computer Music Conference*, pages 64–69, 2012. URL: https://www.charlie-roberts.com/pubs/Gibber_charles_roberts_icmc_2012.pdf.
- [25] Charles Roberts, Matthew Wright, JoAnn Kuchera-Morin, and Tobias Höllerer. Gibber: Abstractions for creative multimedia programming. In *Proceedings of the 22nd ACM International Conference on Multimedia*, MM ’14, page 67–76, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2647868.2654949.
- [26] Felix Roos and Alex McLean. Strudel: Live coding patterns on the web. In *Proceedings of the 7th International Conference on Live Coding*, Utrecht, Netherlands, 2023. doi:10.5281/zenodo.7842142.
- [27] Sonic Pi. Sonic Pi Tutorial, 2026. Accessed: 2026-05-03. URL: <https://sonic-pi.net/tutorial.html>.
- [28] Strudel. Strudel documentation, 2026. Accessed: 2026-04-21. URL: <https://strudel.cc/learn/getting-started/>.
- [29] Tailwind CSS. Tailwind CSS: Utility-first CSS framework, 2026. Accessed: 2026-04-21. URL: <https://tailwindcss.com/>.
- [30] TidalCycles documentation, 2025. URL: <https://tidalcycles.org/docs/>.
- [31] Tone.js Team. Tone.js, 2026. Accessed: 2026-05-03. URL: <https://tonejs.github.io/>.
- [32] TOPLAP. TOPLAP draft manifesto, 2004. Accessed: 2026-05-03. URL: <https://toplap.org/wiki/ManifestoDraft>.
- [33] W3C Audio Working Group. Web Audio API 1.1, 2024. W3C Recommendation. URL: <https://www.w3.org/TR/webaudio-1.1/>.
- [34] Ge Wang and Perry R. Cook. ChucK: A concurrent, on-the-fly audio programming language. In *Proceedings of the International Computer Music Conference*, 2003. URL: <http://hdl.handle.net/2027/spo.bbp2372.2003.055>.
- [35] Elliott Wen, Paul Denny, Andrew Luxton-Reilly, Sean Ma, Bruce Sham, Chenye Ni, Jun Seo, and Yu Yang. Playsemble: Learning low-level programming through interactive games, 2026. URL: <https://arxiv.org/abs/2602.20167>, arXiv:2602.20167, doi:10.48550/arXiv.2602.20167.

- [36] William Yurcik and Hugh Osborne. A crowd of little man computers: Visual computer simulator teaching tools. In *Proceedings of the 2001 Winter Simulation Conference*, pages 1632–1639, 2001. URL: <https://eprints.hud.ac.uk/id/eprint/2533/>, doi:10.1109/WSC.2001.977496.

Appendix A

AI Prompts/Tools

- **Claude Code (Copilot)**: used for in-editor line completion of repetitive or similar code. This did not extend to the creation of whole original functions, only cases where e.g. one word needed refactoring in multiple places.
- **ChatGPT (Codex)**: used to explain hard to find bugs after online searches and code inspection did not reveal any solutions. Prompts followed the lines of “*this thing is happening*, why is that? Find sections of code that might be responsible.”
- **ChatGPT (Codex)**: used to figure out some aspects of CodeMirror and Lezer integration, as I struggled with understanding their documentation for my specific use cases.
- **ChatGPT**: used to initially format Latex tables in this dissertation.

Appendix B

Auxiliary Material

Website deployed on GitHub pages: <https://sd22404.github.io/music-machine>
GitHub repository: <https://github.com/sd22404/music-machine>

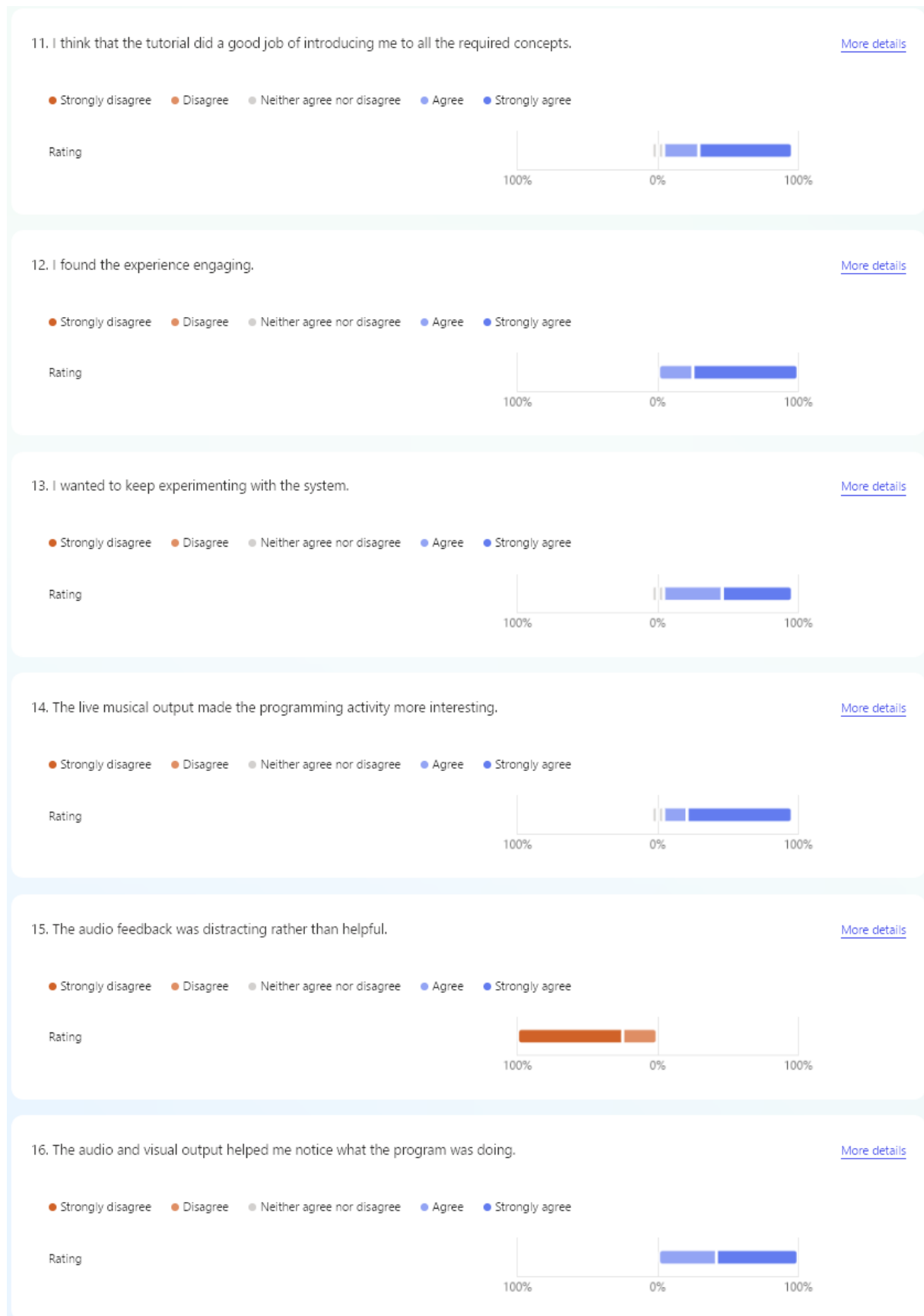


Figure B.1: Results from engagement questions in the final questionnaire.

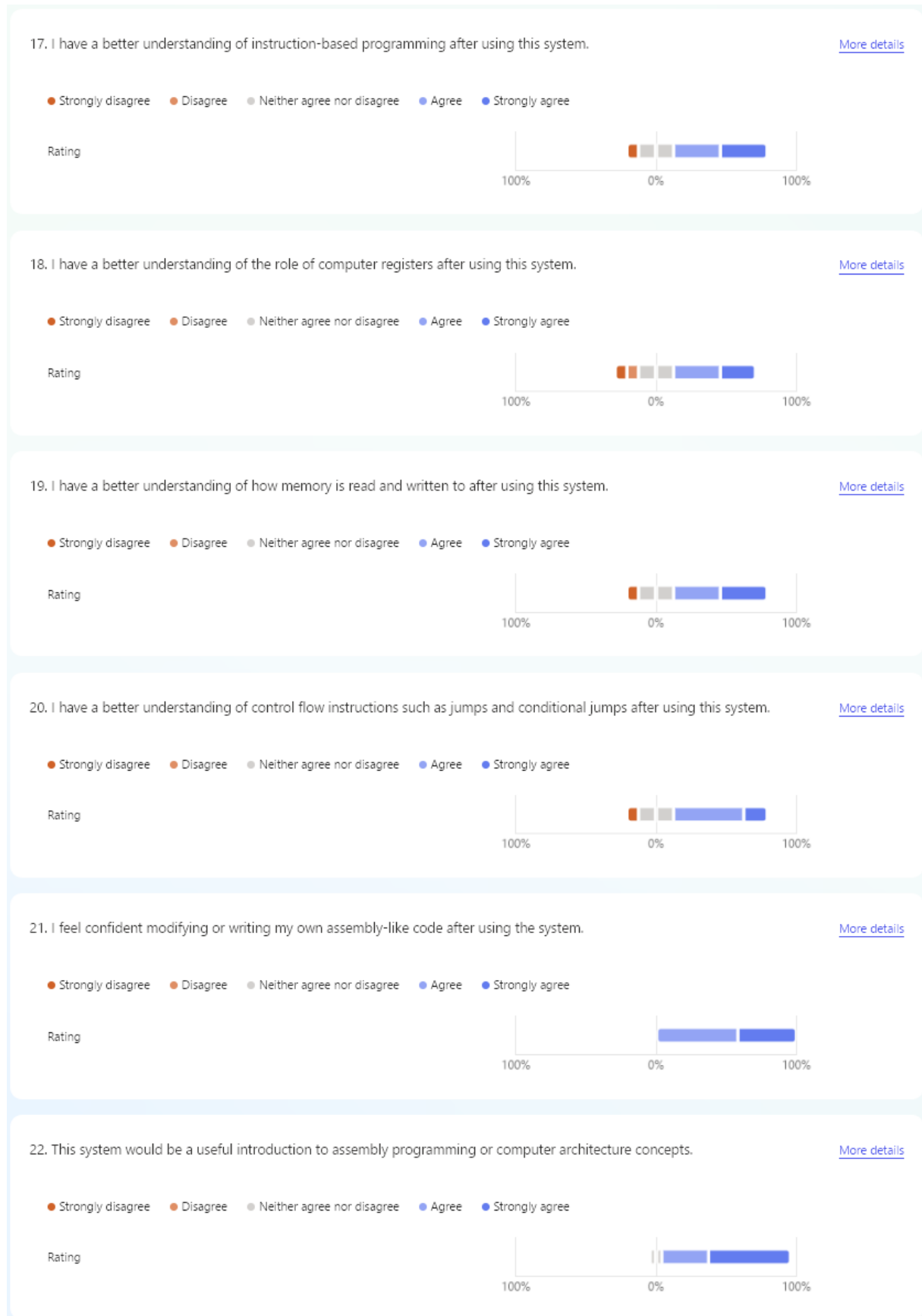


Figure B.2: Results from perceived learning questions in the final questionnaire.